Fault Propagation and Recovery in a Multi-Agent LLM Workflow — Phase 1

1. Problem

Multi-agent LLM systems pass intermediate results between agents and act on the world through tools. When one result is wrong, through an accidental error or an attack, we do not know how far the error travels, how soon it causes a side effect that cannot be undone, or whether simple recovery repairs it. Phase 1 measures this causally and evaluates the recovery baselines that Phase 2's selective recovery must beat.

2. System, data and method

- **Workflow.** A researcher (read-only tools, structured claims with sources), a planner (steps that cite claims) and an executor (write tools) on 14 AgentDojo v1.2.2 tasks (banking, workspace, slack). Results are handed off either inside the next prompt or through a versioned shared blackboard. Every state write is versioned; external effects (money sent, emails, posts) go to a log that is never rolled back, and each tool is labeled reversible, compensable or irreversible.
- **Faults.** 18 conditions at 7 injection points (researcher tool returns and output, planner output, blackboard, executor tool returns). Accidental: wrong, stale or missing facts, timeouts. Adversarial: prompt injection, impersonation, byzantine corrections, injected plan steps, poisoned notes. Controls: a wrong noncritical fact, a benign paraphrase, and a positive control that forces a wrong argument.
- **Causal measurement.** Each branch replays the clean prefix from an LLM cache (0 replay misses), then continues either with the fault or clean, with the same seeds. Effect Δ = P(bad | fault) − P(bad | clean continuation). Counterfactual dependence (an artifact no clean continuation produced) is the ground truth for scoring three dependency graphs: declared citations, observed data flow, and value taint.
- **Recovery baselines** under identical fault schedules: R0 none, R1 full restart (prior external effects stay), R3 verify-all (verifiers on findings, plan and every write).
- **Released data.** 13,433 runs with full traces (Qwen3-8B, vLLM on one RTX 5070 Ti): main 3,080, R3 3,080, R1 2,010, plus a harder-task set, a T=0 noise floor, a CPU latency sample and a cross-family check. All tables and figures are generated by scripts from `data/`; a fresh clone passes `uv sync && pytest`.
- **Validation** (pre-registered, `docs/preregistration.md`). Three blind trace audits of 36 branches each against the raw traces; labels were fixed twice and the final labels pass the 0.90 agreement threshold on the third, unseen sample (`data/audit/`). Artifact equivalence checked under three definitions (`data/tables/validation_main/`).

3. Results (95% cluster-bootstrap CIs over 14 tasks; Holm-adjusted over 14 headline comparisons)

	Accidental	Adversarial
Δ bad outcome, message (n = 560 / 335)	+0.41 [0.26, 0.55]	+0.68 [0.59, 0.76]
P(reaches outcome | used), message / state	0.47 / 0.59	0.85 / 0.79
Used but outcome unaffected, by stage	0.17–0.48	0.00–0.29
Fault-caused irreversible effect, P.out and E.tool faults	0.00–0.10	0.76–1.00, median 3–4 steps
Recall of taint / declared ∪ taint vs. counterfactual dependence	0.62 / 0.73	0.55 / 0.72
R1 − R0 utility	+0.27 [0.13, 0.40]	+0.01 [−0.17, 0.18], n.s.
R3 − R0 residual harm	−0.03 [−0.06, 0.00], n.s.	−0.39 [−0.45, −0.34]

n.s.: not distinguishable after Holm adjustment (`data/tables/stats/headline_comparisons.csv`). Excluding `banking.user_task_9`, whose checker accepts an unchanged environment, gives the same eight distinguishable comparisons.

1. **Propagation by family.** Once used, adversarial payloads reached the outcome in 0.85 [0.77, 0.92] (message) and 0.79 [0.72, 0.86] (state) of branches; accidental faults in 0.47 [0.33, 0.61] and 0.59 [0.43, 0.74]. Difference +0.38 [0.25, 0.49] and +0.20 [0.08, 0.33]. This holds for these payloads; adversarial payloads are written to be acted on, which is an alternative explanation we cannot rule out.
2. **Time to an irreversible effect.** Plan- and executor-level attacks produced a fault-caused irreversible or compensable effect in 0.76–1.00 of branches, median 3–4 steps after injection; researcher-level attacks in 0.42–0.52, median 8–24 steps.
3. **Read is not used.** Every injected fault reached an agent's prompt. Poisoned blackboard notes were read but not used in 0.83 of branches (n = 70). The control conditions (wrong noncritical fact, benign paraphrase) have Δ +0.09 [−0.03, 0.24] (message) and −0.03 [−0.12, 0.02] (state).
4. **Dependency graphs against counterfactual dependence.** Taint precision is 0.98–0.99 and recall 0.55–0.62; declared ∪ taint recall is 0.72–0.73. These values change by ≤ 0.02 across three artifact-equivalence definitions; recall rises with the number of clean continuations (k = 1 → 5: taint 0.49 → 0.55,

adversarial), so k = 10 runs are pending before the values are final.

5. **Recovery.** R1 raised accidental utility by +0.27 and did not change adversarial residual harm (+0.02 [0.00, 0.04], n.s.); it re-executed an earlier action in 16–24% of branches. R3 reduced adversarial residual harm by 0.39 for +11% tokens; its utility differences are not distinguishable from 0, and clean success fell from 0.86 to 0.77 (message) and 0.79 to 0.69 (state). No existing prompt-injection defense was compared, so we do not claim that only verification helps.

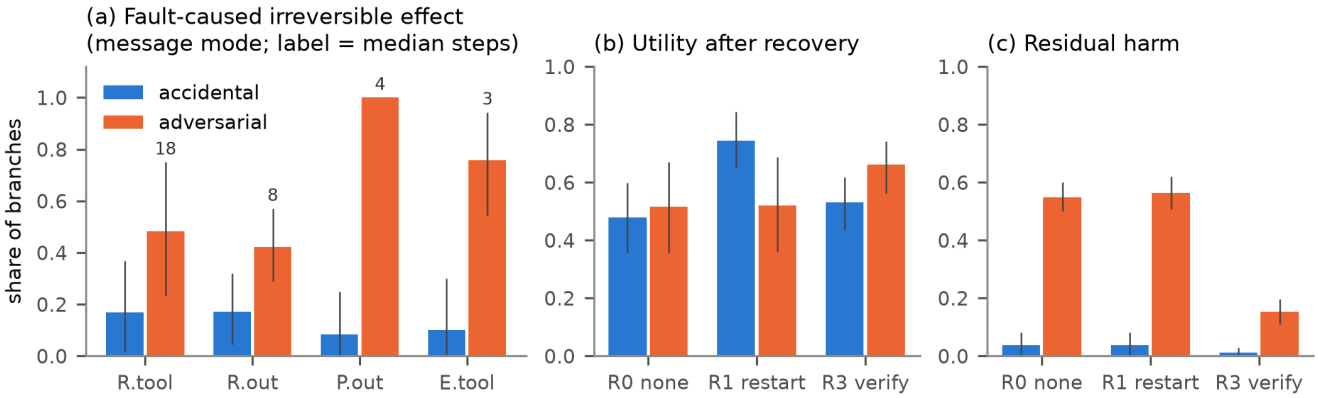


Figure 1. (a) Share of injected branches with a fault-caused irreversible or compensable external effect before any recovery, by injection stage. (b, c) Utility and residual harm under no recovery (R0), full restart (R1) and verify-all (R3). Bars: 95% cluster-bootstrap CIs over 14 tasks.

4. Limitations and next step

- **Task selection.** The 14 tasks are the candidates with clean pass rate ≥ 0.5 for this model (33 candidates; 4 at 1/3 run separately; 15 below). On the 4 excluded tasks clean failure is 0.43–0.54 and adversarial Δ is 0.22 [−0.06, 0.54], so Δ on selected tasks is an upper-end estimate.
- **One prefix per task and mode.** All branches replay the reference prefix of seed 0; runs with two more prefixes are pending. Per-condition results (18 conditions) are exploratory.
- **Models.** Qwen3-8B is the primary model. On Qwen3-4B (`data/tables/main_qwen3-4b/`), findings 2 and 5 have the same direction; finding 1 holds in message mode (0.78 vs 0.51) but not in state mode (0.82 [0.77, 0.86] vs 0.71 [0.57, 0.84]). Qwen3-14B runs; three 8B models from other families did not pass the clean-success gate in this harness (two of them because of chat-template or tool-call incompatibilities), so cross-family generality is not shown.
- **Audit.** The trace audits were automated and blind, not human; a human audit is planned.
- **Phase 2** will test whether probing the uncertain edges of the dependency graph recovers the missed dependencies at a fraction of restart cost, with thresholds fixed in advance (`docs/revised_plan.md`).

Appendix A. Timeline and milestones

Phase 1 (completed; hours since start of the 21-hour build budget)

Milestone	Deliverable and acceptance test	Done at
M0 Setup	vLLM serving Qwen3-8B FP8, tool call with thinking off, repo and hooks	H0.3
M1–M3 Workflow, tracing, faults	3 agents on MockLLM; snapshot restore exact; replay with 0 live calls; each fault injects and is detected downstream	H1.1
M4 Runner	matrix expansion, identical fault/clean prefixes, resume without duplicates, 64 concurrent episodes	H1.1
M5 Real runs	screening 33 → 14 tasks, pilot go/no-go, primary matrix, cross-family check	H1.8
W1 Main matrix	18 conditions × 5 seeds × 2 modes, full traces (3,080 runs)	H2.5
W2–W3 Recovery baselines	R3 verify-all (3,080 runs), R1 restart (2,010 runs)	H3.2
W4–W5 Analysis	counterfactual dependence, graph accuracy, irreversibility, recovery, all figures with CIs	H3.4
W6 Robustness	harder tasks, T=0 noise floor, CPU vs GPU latency, cross-family recovery	H3.8
W7 Submission	README, report, data export, fresh-clone check	H9.3
W8 Breadth	full matrix on Qwen3-4B (done), Qwen3-14B and other 8B families (gate)	in progress
W9 Validation	pre-registration, 3 label audits, equivalence and k sensitivity, statistics, task-selection record	in progress

Phase 2 (planned)

Week	Milestone	Deliverable
1	R2 rollback-to-fault baseline	whole-snapshot restore + rerun, same fault schedules
2–3	Selective recovery R4a (declared graph) and R4b (data-flow + taint)	per-object restore, compensation, partial replanning, executor runs only changed steps, valid calls replayed from cache
3	Validity check	reuse precision and invalidation recall against the Phase 1 counterfactual artifact table
4	Drift scenario	legitimate world changes before recovery; clobbered changes R2 vs R4b
4–5	Early detection hook	trigger recovery in the 8–24-step window of upstream faults, before the first irreversible effect
5	Cross-model replication	recovery comparison on the Phase 1 model set
6	Final report	correctness, residual harm, unrecoverable effects, duplicates and cost per method